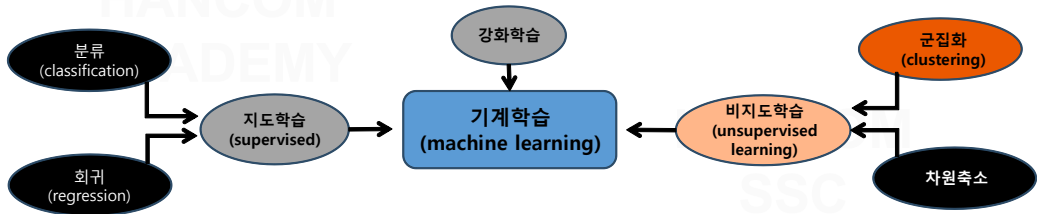


Chapter 07 .군집

- K- means 알고리즘
- 군집 예제

군집화

- : 비지도학습의 일종으로 데이터의 특징만으로 비슷한 데이터들끼리 모아 군집된 클래스로 분류
- : 비지도학습은 정답 없이 특징만 가지고 있는 데이터로 모델을 구현
- : **K-means(K 평균)알고리즘**은 데이터 간의 거리를 사용해(평균) 가까운 거리에 있는 데이터끼리 k개의 클래스로 묶는 강력한 군집화 알고리즘



* 군집화 목적

- 비지도학습 방법에 속하여 레이블(정답)이 없는 데이터를 특정 클러스터끼리 묶는(군집) 목적
- 분류와 달리 어떤 레이블에 속할지 예측하기 보다는 어떤 클러스터에 군집화 될 수 있구나 정도만 파악하는 기능이며 이 정보가 필요할 경우 군집화 알고리즘을 사용 (레이블이 달려 있지 않은 입력 데이터에 레이블을 달아주는 역할 수행)

* 분류 목적

- 지도학습 방법에 속하여 정답(레이블)이 주어졌을 때 레이블을 기반으로 특정 데이터를 예측 분류하는 목적
- 모델이 예측한 레이블과 실제 레이블을 비교하여 모델의 성능 판단

K-means 알고리즘 원리

: 데이터간의 거리를 계산하기 위해 수치화된 데이터 사용

1) 군집의 개수(K) 설정

- 몇 개 클러스터로 군집화 할지 **군집 개수 지정** (default : 8) ,
- 알고리즘 성능을 평가하는 매우 중요한 지표,
대표적으로 Elbow Method 방법론 활용 군집의 개수 설정

2) 초기 중심점 설정

- **초기 클러스터 중심점 설정** 방법(3가지)
: 무작위로 설정, 최초 중심값 수동 부여, kmean++
- default : **kmean++ 알고리즘**

3) 데이터를 군집에 할당

- 거리 상 가장 가까운 클러스터 중심점으로 모든 데이터를
순회하면서 **군집(중심점)에 할당(소속시킴)**

4) 중심점 재 설정(갱신)

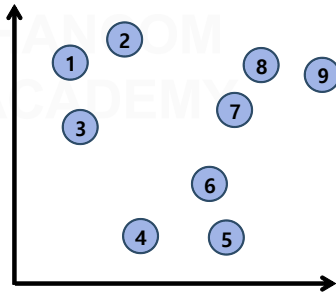
- 모든 데이터의 군집(중심점) 배정 끝나면 군집의 **중심점**
(Centroid)을 그 군집의 소속된 데이터들의 가장 중간(거리
평균)에 위치한 지점으로 **재 설정**

5) 데이터를 군집(중심점)에 재 할당

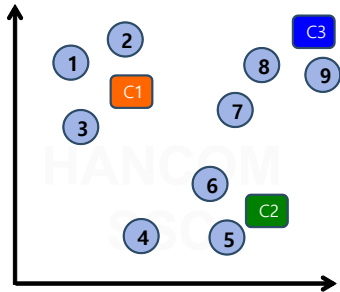
- step 3 번 처럼 모든 데이터를 순회하면서 **군집(중심점)에
재 할당(소속시킴)**, 더 이상 중심점 이동 없을 때까지
step4 와 step5 반복

중심점 위치
변화 없을 때
까지 반복

K-means 알고리즘 원리(그림)

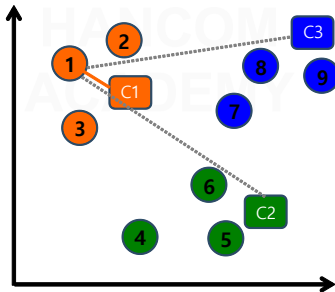


수치화된 임의의 데이터
배치 그래프

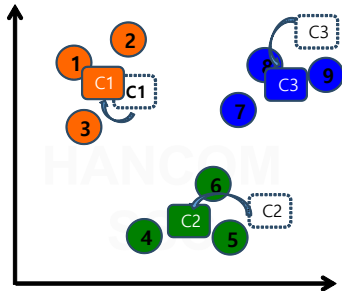


- 1) 군집의 개수(3개) 설정
- 2) 초기 중심점 설정

K-means 알고리즘 원리(그림)

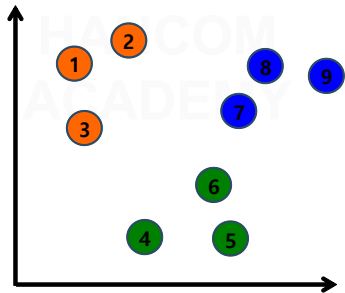


3) 거리 측정하여 각 데이터를 가까운 중심점에 배정



4) 중심점 재 설정(갱신)

K-means 알고리즘 원리(그림)



5) 데이터를 군집(중심점)에 재 할당(배정)

4)~5) step 반복, 더 이상
중심점 이동 없으면 종료

kmean++ 알고리즘

- 모든 중심점을 랜덤하게 설정하는 단점을 보완하여 랜덤하게 최초 데이터 포인트 지점을 중심점 하나로 설정하고 해당 지점에서 가장 먼 데이터 포인트를 두번째 클러스터 중심점, 그리고 기존의 클러스터 중심점에서 가장 먼 데이터 포인트를 그 다음 클러스터 중심으로 설정

예제 1) K-means 과일 데이터 군집화

```
import numpy as np
np.set_printoptions(threshold=np.inf) # 넘파이 데이터 생략 없이 모두출력

fruits = np.load('fruits_300.npy') # 300개 과일 이미지 넘파이 배열 샘플 데이터
print(len(fruits)) # 총 300개 샘플 : 3차원배열 ( 샘플개수, 너비, 높이 )
# 한개 과일 샘플에 너비 * 높이 pixel = 100 * 100
print(len(fruits[1][0])) # 너비 길이
print(len(fruits[1][0][0])) # 높이 길이
```

k-평균 모델훈련을 위해 2차원배열로 변환

```
fruits_2D = fruits.reshape(-1,100*100)
print(fruits_2D.shape)
```

(300, 10000)

k-평균 모델 훈련

```
from sklearn.cluster import KMeans
```

n_clusters : 클러스터 개수 지정

init='k-means++' 디폴트 설정

```
km = KMeans(n_clusters=3, random_state=42)
```

비지도학습으로 fit() 메서드 타깃 데이터 사용하지 않음

```
km.fit(fruits_2D)
```

```
labels = km.fit_predict(fruits_2D)
# 클러스터 중심점 계산 + 클러스터 중
# 심점에 가까운 라벨 할당 및 반환
print('labels: ', labels)
```

★ 대체 가능

예제 1) K-means 과일 데이터 군집화

군집 결과는 KMeans 클래스 labels 속성에 저장

labels 길이는 샘플 개수 (300개), 각 샘플이 어떤 레이블에 해당되는지 나타냄

n_clusters=3 지정으로 labels 배열의 값은 0,1,2 중 하나

```
print(km.labels_)
```

[illegible]

레이블 0,1,2로 모은 샘플 개수 확인

```
print(np.unique(km.labels_, return_counts=True))
```

```
(array([0, 1, 2]), array([111, 98, 91], dtype=int64))
```

(0 : 1117H)

(1 : 98개)

예제 1) K-means 과일 데이터 군집화

각 클러스터(0, 1, 2) 이미지 그림으로 출력

```
import matplotlib.pyplot as plt
```

```
def draw_fruits(arr, ratio = 1):
```

```
    n = len(arr) # 함수 호출시 넘어온 각 클러스터 샘플 개수
```

```
    # 한줄에 10개 이미지 출력위해 샘플 개수를 10으로 나눠 행 개수 계산
```

```
    rows = int( np.ceil(n/10) ) # ceil() : 입력값보다 같거나 큰 정수중 가장 가까운값 반환
```

```
    # 행이 1이면 열의 개수는 샘플 개수, 그 이상의 행이면 10 열
```

```
    cols = n if rows < 2 else 10
```

```
    # squeeze=False ==> 배열을 압축하지 말고 rows, cols 지정 형태 그대로 형성
```

```
    # 즉, (1,2) 일때 (2,)처럼 압축하지 말고 (1,2) 형태로 형성
```

```
    fig, axs = plt.subplots(rows, cols,
```

```
                            figsize=(cols*ratio, rows*ratio), squeeze=False)
```

```
    for row in range(rows):
```

```
        for col in range(cols):
```

```
            if row*10 + col < n: # n개 까지만 그림
```

```
                axs[row,col].imshow(arr[row*10+col], cmap='gray_r')
```

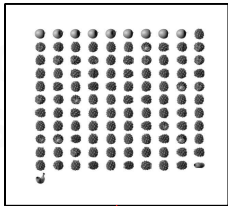
```
                axs[row,col].axis('off') # 모든 축과 눈금, 라벨 지워지고 그래프(이미지)만 출력
```

```
    plt.show()
```

```
draw_fruits( fruits[km.labels_==0] ) # 불린배열 인덱싱 활용 0 클러스터 샘플 추출 (파인애플)
```

```
#draw_fruits(fruits[km.labels_==1]) # 불린배열 인덱싱 활용 1 클러스터 샘플 추출 (바나나)
```

```
#draw_fruits(fruits[km.labels_==2]) # 불린배열 인덱싱 활용 2 클러스터 샘플 추출 (사과)
```



예제 2) K-means 최적 K 값 찾기 - 엘보우 방법 활용

```
import numpy as np
np.set_printoptions(threshold=np.inf)

fruits = np.load('fruits_300.npy')
print(len(fruits)) # 총 300개 샘플 : 3차원배열 ( 샘플개수, 너비, 높이 )
# 한개 과일 샘플에 너비 * 높이 pixel = 100 * 100
print(len(fruits[:1][0]))
print(len(fruits[:1][0][0]))

# k-평균 모델훈련을 위해 2차원배열로 변환
fruits_2D = fruits.reshape(-1,100*100)
print(fruits_2D.shape)
```

예제 2) K-means 최적 K 값 찾기 - 엘보우 방법 활용

k-평균 모델 훈련

```
from sklearn.cluster import KMeans
```

```
import matplotlib.pyplot as plt
```

```
inertia = []
```

```
for k in range(2,7):
```

```
    km = KMeans(n_clusters=k, random_state=42)
```

```
    km.fit(fruits_2D)
```

```
    inertia.append( km.inertia_ ) # 이너서 : 클러스터 중심과 클러스터에 속한 샘플 사이의 거리의 제곱 합
```

이너서가 작을수록 중심점에 잘 군집됐다고 판단하며

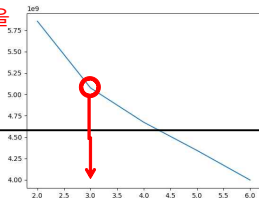
클러스터의 개수를 늘려도 밀집된 정도가 개선되지 않은 시점(엘보우 방법)을

최적의 k 로 판단

```
plt.plot(range(2,7), inertia)
```

```
plt.show()
```

이너서 출력 결과
꺼이는 지점이 최적 K로
판단 (K = 3 이 최적)



Thank You

HANCOM

Template Visual Guide, version 1.0

© Hancorn Inc. / Pangyo, February 2019